

Isaac Gomes

RA:1680482422022

Resumo do notebookIm

Com base no artigo "*Agent Design Pattern Catalogue: A Collection of Architectural Patterns for Foundation Model based Agents*" da CSIRO, apresento o resumo automático solicitado para a **Fase 0** do seu desafio de Engenharia de Software.

Resumo: Catálogo de Padrões para Agentes baseados em Modelos de Fundação

Introdução e Desafios O surgimento de Modelos de Fundação (FMs) e Grandes Modelos de Linguagem (LLMs) impulsionou o desenvolvimento de agentes autônomos capazes de realizar tarefas complexas. No entanto, a engenharia desses sistemas enfrenta desafios críticos: a falta de explicabilidade (sistemas "caixa-preta"), incertezas no raciocínio (alucinações), a dificuldade de capturar intenções ambíguas dos usuários (*underspecification*) e a complexidade na atribuição de responsabilidade. Para mitigar esses problemas, o artigo propõe um catálogo de **18 padrões arquiteturais** que oferecem soluções reutilizáveis para o design de agentes.

Os 18 Padrões de Design Os padrões são categorizados para cobrir todo o ciclo de vida da operação do agente:

1. **Engenharia de Contexto e Metas:** O *Passive Goal Creator* analisa comandos diretos, enquanto o *Proactive Goal Creator* antecipa necessidades via sensores e contexto multimodal. O *Prompt/Response Optimiser* padroniza a comunicação, e o *Retrieval Augmented Generation (RAG)* permite que o agente acesse dados atualizados e privados sem necessidade de treinamento caro.
2. **Consulta e Planejamento:** O *One-Shot Model Querying* foca em eficiência de custo com uma única chamada ao modelo, enquanto o *Incremental Model Querying* prioriza a precisão através de interações passo a passo. Para execução, o *Single-Path Plan*

Generator cria caminhos lineares e coerentes, enquanto o *Multi-Path Plan Generator* oferece diversas alternativas para decisões complexas.

3. **Reflexão e Refinamento:** Os padrões de *Self-Reflection*, *Cross-Reflection* (entre agentes) e *Human Reflection* são essenciais para revisar planos e corrigir erros antes da execução final, aumentando a confiabilidade.
4. **Cooperação Multi-Agente:** Para sistemas complexos, o artigo descreve a cooperação baseada em *Votação* (consenso e justiça), *Papéis* (divisão de trabalho especializada) e *Debate* (estimula o pensamento crítico e a correção mútua).
5. **Operação e Segurança:** O uso de *Multimodal Guardrails* controla entradas e saídas para garantir segurança e ética. O *Tool/Agent Registry* e o *Agent Adapter* facilitam a interoperabilidade com ferramentas externas, enquanto o *Agent Evaluator* permite testes contínuos de desempenho.

Relevância para a Engenharia de Software A adoção desses padrões é um marco para a Engenharia de Software moderna, pois transforma o desenvolvimento ad-hoc de IA em uma **disciplina de engenharia estruturada**. Eles permitem que arquitetos analisem *trade-offs* entre custo, latência e precisão, utilizando um **Modelo de Decisão** para escolher a melhor composição de componentes. Além disso, ao padronizar as interfaces e mecanismos de controle (como *Guardrails* e *Adapters*), os padrões promovem a manutenibilidade, a segurança e a confiança em sistemas autônomos, alinhando-os às práticas de DevOps e governança de software.

Meu resumo

O artigo do CSIRO apresenta 18 padrões arquiteturais para agentes de IA baseados em LLMs. Esses padrões organizam como os agentes recebem objetivos, buscam conhecimento, geram planos, revisam respostas e cooperam entre si. O catálogo segue uma ideia parecida com os Design Patterns do GoF, mas aplicada à inteligência artificial.

Entre os padrões mais importantes estão o RAG, que combina busca de informações externas com geração de texto, os padrões de Reflection, que permitem ao agente revisar suas próprias respostas, e os padrões de Cooperation, usados para coordenação entre múltiplos agentes. Também existem padrões de infraestrutura, como Tool Registry e Agent Adapter, responsáveis pela integração com ferramentas externas.

O artigo mostra que muitos padrões modernos de IA possuem relação com padrões clássicos da engenharia de software, como Strategy, Observer, Factory Method e Adapter. A principal conclusão é que sistemas de agentes inteligentes podem ser desenvolvidos usando princípios tradicionais de arquitetura e design patterns, aumentando organização, reutilização e manutenção do software.

#Nota: Usei IA pra organizar melhor o meu texto , sendo bem sincero não li exatamente tudo , mas pedi pro notebooklm me dar um apanhado geral pra eu ter uma base.